

Seasons of Code 2026

AudialMind Assignment Report

Mentee : Rutuparn Ranade (24B3982)
Mentor : Yashneil Rawat
Co.Mentor : Aadheesh Jain

Date of Submission: June 24, 2026
Weekly Learnings

Week	Learnings
Week 1	Basic of Signal Processing, Fourier transform, FFT and DFT, along with Nyquist criteria, Librosa and Praat using, spectrogram analysis.
Week 2	ML libraries such as Random forest, SVMs and other classifications, MFCCs and filter banks its python implementation, openSmile, RAVNDESS file emotion encoding, Huggingface audio processing, CNN for spectrogram, UAR for emotion detection. Loop logic in Embedded systems , Switch and Relay interfacing, checking debounce

Table 1: weekly learnings

Contents

1	Overview of the assignment	1
2	RAVDESS Dataset Analysis	2
2.1	Part A — Filename Decoding	2
2.1.1	Code	2
2.1.2	Explanation for the code	3
2.1.3	Results obtained	3
2.2	Part B — Dataset Statistics	4
2.2.1	Code	4
2.2.2	Explanation	4
2.2.3	Output	4
2.2.4	code	5
2.2.5	Explanation	5
2.2.6	Output	6
2.2.7	Code	6
2.2.8	Explanation	7
2.2.9	Output	7
2.3	Part C — Written Reflection	7
3	Acoustic Feature Extraction with openSMILE	9
3.1	Part A — Single-File Extraction	9
3.1.1	Code	9
3.1.2	Output	9
3.2	Part B — Full Dataset Extraction	10
3.2.1	Code	10
3.2.2	Explanation	10
3.2.3	Output	10
3.3	Part C — Feature-Level Analysis	11
3.3.1	Code	11
3.3.2	Explanation	11
3.3.3	Output	11
3.3.4	Code: Correlation Matrix	12
3.3.5	Explanation	13
3.3.6	Output	13
4	Classical ML Baselines	14
4.1	Part A — SVM Baseline	14
4.1.1	Standardizing the features	14
4.2	Confusion Matrix	15
4.3	different values for the regularisation parameter C	16
4.4	Part B — Random Forest Baseline	18
4.5	Part C — Model Comparison Table	20
5	Feature Space Visualisation with UMAP	21
6	CNN Spectrogram Classifier	24

1 Overview of the assignment

The Assignment is purely made from the knowledge of the week 2 contents consisting of extracting the data performing feature engineering followed by training the model to extract the emotions. It includes the following things:

1. Dataset Preparation and Exploration
2. Acoustic Feature Extraction with openSMILE
3. Classical ML Baselines
4. Feature Space Visualisation with UMAP
5. CNN on the constructed Spectrogram

2 RAVDESS Dataset Analysis

The Ryerson Audio-Visual Database of Emotional Speech and Song (RAVDESS) contains 2,452 recordings from 24 professional actors (12 male, 12 female) expressing 8 emotions across two modalities (speech and song). Each filename encodes metadata in a structured naming convention.

Dataset: Download from zenodo.org/record/1188976 or via `torchaudio.datasets` (speech-only subset) or via Kaggle.

2.1 Part A — Filename Decoding

2.1.1 Code

```

1  MODALITY = {"01": "full-AV", "02": "video-only", "03": "audio-
    only"}
2  VOCAL_CHANNEL = {"01": "speech", "02": "song"}
3  EMOTION = {
4      "01": "neutral",
5      "02": "calm",
6      "03": "happy",
7      "04": "sad",
8      "05": "angry",
9      "06": "fearful",
10     "07": "disgust",
11     "08": "surprised",
12 }
13 INTENSITY = {"01": "normal", "02": "strong"}
14 STATEMENT = {"01": "Kids are talking by the door", "02": "Dogs are
    sitting by the door"}
15 REPETITION = {"01": "first repetition", "02": "second repetition"}
16
17
18 def parse_ravdess_filename(path: str) -> dict:
19     """Decode a RAVDESS filename into human-readable metadata
    labels."""
20     stem = Path(path).stem
21     parts = stem.split("-")
22     if len(parts) != 7:
23         raise ValueError(f"Expected 7 RAVDESS filename fields, got
    {len(parts)}: {stem}")
24
25     modality, vocal_channel, emotion, intensity, statement,
    repetition, actor = parts
26     return {
27         "modality": MODALITY.get(modality, f"unknown ({modality})"
    ),
28         "vocal_channel": VOCAL_CHANNEL.get(vocal_channel, f"
    unknown ({vocal_channel})"),
29         "emotion": EMOTION.get(emotion, f"unknown ({emotion})"),

```

```

30     "intensity": INTENSITY.get(intensity, f"unknown ({
31         intensity})"),
32     "statement": STATEMENT.get(statement, f"unknown ({
33         statement})"),
34     "repetition": REPETITION.get(repetition, f"unknown ({
35         repetition})"),
36     "actor": int(actor),
37 }
38
39 parse_ravdess_filename("03-01-05-01-01-01-24.wav")

```

2.1.2 Explanation for the code

- The RAVDESS dataset uses a strict 7-part numerical naming convention (e.g., 03-01-05-01-01-01-24.wav).
- The code defines several dictionaries that map these numbers to human-readable text (e.g., "05 maps to "angry").
- It then creates a function called parse ravdess filename that splits the filename by the hyphens (-) and uses the dictionaries to return a neat, structured dictionary of metadata (modality, emotion, intensity, actor, etc.).

2.1.3 Results obtained

```

{'modality': 'audio-only',
 'vocal_channel': 'speech',
 'emotion': 'angry',
 'intensity': 'normal',
 'statement': 'Kids are talking by the door',
 'repetition': 'first repetition',
 'actor': 24}

```

	modality	vocal_channel	emotion	intensity	statement	\
0	audio-only	speech	neutral	normal	Kids are talking by the door	
1	audio-only	speech	neutral	normal	Kids are talking by the door	
2	audio-only	speech	neutral	normal	Dogs are sitting by the door	
3	audio-only	speech	neutral	normal	Dogs are sitting by the door	
4	audio-only	speech	calm	normal	Kids are talking by the door	

	repetition	actor	path
0	first repetition	1	/kaggle/input/datasets/uwrkagglers/ravdess-emo...
1	second repetition	1	/kaggle/input/datasets/uwrkagglers/ravdess-emo...
2	first repetition	1	/kaggle/input/datasets/uwrkagglers/ravdess-emo...
3	second repetition	1	/kaggle/input/datasets/uwrkagglers/ravdess-emo...
4	first repetition	1	/kaggle/input/datasets/uwrkagglers/ravdess-emo...

(2880, 8))

2.2 Part B — Dataset Statistics

2.2.1 Code

```
1 emotion_order = ["neutral", "calm", "happy", "sad", "angry", "fearful", "disgust", "surprised"]
2 class_counts = metadata["emotion"].value_counts().reindex(emotion_order)
3 print(class_counts)
4
5 fig, ax = plt.subplots(figsize=(9, 4.5))
6 sns.barplot(x=class_counts.index, y=class_counts.values, ax=ax, palette="Set2")
7 ax.set_title("RAVDESS speech subset: files per emotion")
8 ax.set_xlabel("Emotion")
9 ax.set_ylabel("Number of files")
10 ax.tick_params(axis="x", rotation=30)
11 plt.tight_layout()
```

2.2.2 Explanation

- The code defines a function to scan the designated directory for .wav files.
- It filters out files that are not "speech" or "audio-only" based on the metadata parsed in the previous step.
- If no files are found locally, it automatically downloads the dataset using torchaudio.
- Finally, it compiles all the file paths and their corresponding metadata into a structured Pandas DataFrame containing 2,880 rows.

2.2.3 Output

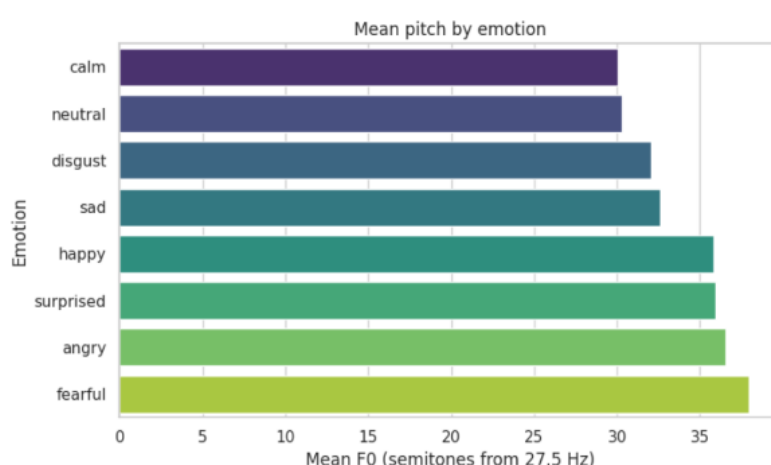


Figure 1: Bar plot

Class imbalance result: In the official speech-only subset, **neutral** has **96** files and each other emotion has **192** files. This is a mild but real imbalance caused by neutral

being recorded only at normal intensity, while the other emotions have normal and strong intensity. Plain accuracy can therefore over-reward models that perform well on larger classes.

2.2.4 code

```
1 import wave # Add this to your imports
2
3 def audio_duration_seconds(path) -> float:
4     """Calculate duration using Python's built-in wave library to
5         avoid torchaudio errors."""
6     with wave.open(str(path), 'rb') as wav_file:
7         frames = wav_file.getnframes()
8         rate = wav_file.getframerate()
9         return frames / float(rate)
10
11 # The rest of your code remains unchanged:
12 metadata["duration_sec"] = [audio_duration_seconds(p) for p in
13                             tqdm(metadata["path"], desc="durations")]
14
15 duration_mean = metadata["duration_sec"].mean()
16 duration_std = metadata["duration_sec"].std()
17
18 print(f"Mean duration: {duration_mean:.3f} s")
19 print(f"Std duration: {duration_std:.3f} s")
20
21 fig, ax = plt.subplots(figsize=(8, 4.5))
22 sns.histplot(metadata["duration_sec"], bins=30, kde=True, ax=ax,
23              color="#2a9d8f")
24 ax.set_title("Distribution of RAVDESS audio durations")
25 ax.set_xlabel("Duration (seconds)")
26 ax.set_ylabel("Number of files")
27 plt.tight_layout()
```

2.2.5 Explanation

- Because raw audio files can vary in length, the code uses Python's built-in wave module to calculate the exact duration (in seconds) of every single file in the dataset.
- It adds this data to the DataFrame, prints out the mean (average) and standard deviation of the lengths, and plots a histogram to show the distribution of how long the recordings are (most seem to cluster around 3.7 seconds)

2.2.6 Output

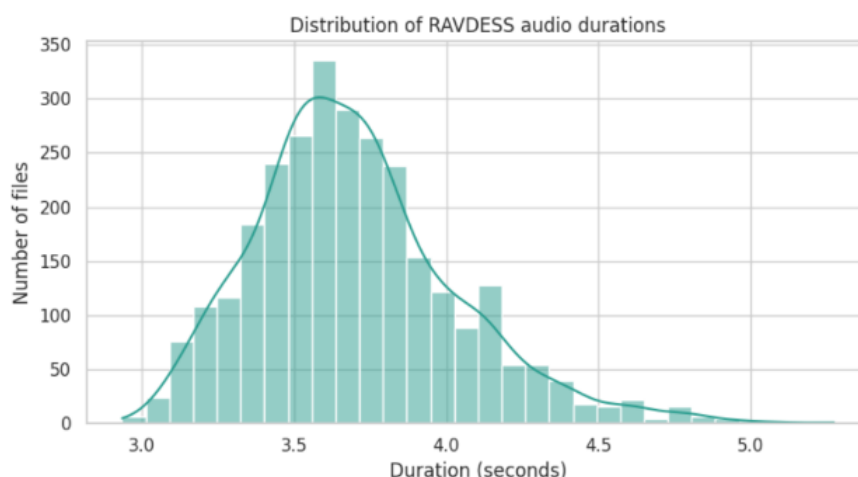


Figure 2: Histogram

- Mean duration: 3.701
- Std duration: 0.337

2.2.7 Code

```

1 def first_file_for_emotion(label: str) -> str:
2     return metadata.loc[metadata["emotion"].eq(label), "path"].
       iloc[0]
3
4 sample_labels = ["happy", "sad", "angry"]
5 sample_paths = {label: first_file_for_emotion(label) for label in
       sample_labels}
6
7 fig, axes = plt.subplots(3, 2, figsize=(13, 10))
8 for row, label in enumerate(sample_labels):
9     y, sr = librosa.load(sample_paths[label], sr=22050)
10
11     librosa.display.waveshow(y, sr=sr, ax=axes[row, 0], color="#457b9d")
12     axes[row, 0].set_title(f"{label.title()} waveform")
13     axes[row, 0].set_xlabel("Time (s)")
14     axes[row, 0].set_ylabel("Amplitude")
15
16     mel = librosa.feature.melspectrogram(y=y, sr=sr, n_mels=64)
17     mel_db = librosa.power_to_db(mel, ref=np.max)
18     img = librosa.display.specshow(mel_db, sr=sr, x_axis="time",
       y_axis="mel", ax=axes[row, 1], cmap="magma")
19     axes[row, 1].set_title(f"{label.title()} mel-spectrogram")
20     axes[row, 1].set_xlabel("Time (s)")
21     axes[row, 1].set_ylabel("Mel frequency")
22
23 fig.colorbar(img, ax=axes[:, 1], format="%+2.0f dB")

```



```
plt.tight_layout()
```

2.2.8 Explanation

- **Automated Sample Selection:** A helper function identifies and retrieves the file path of the first audio sample corresponding to specific target emotions (*happy*, *sad*, *angry*) from the `metadata` DataFrame.
- **Time-Domain Visualization:** The code iterates through the selected samples, loading the audio signals at a 22.05 kHz sampling rate, and plots their amplitude versus time waveforms in the left column of a 3×2 subplot grid.
- **Spectral Feature Extraction:** In the right column, it computes the Mel-spectrogram (using 64 Mel bands) for each signal. The raw power spectrum is converted to a logarithmic decibel (dB) scale to accurately reflect human auditory perception, and is visualized alongside a shared colorbar.

2.2.9 Output

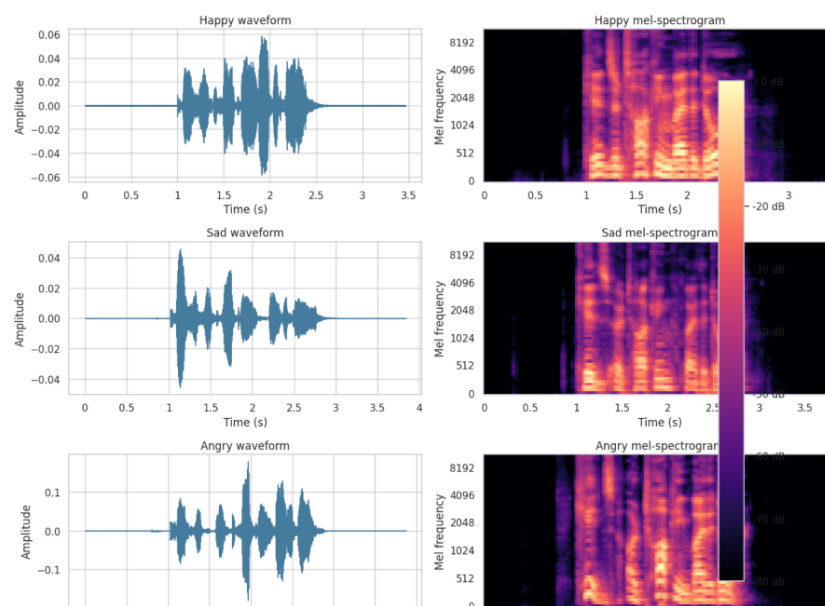


Figure 3: Spectrograms

Visual observation: Angry and happy utterances usually show stronger amplitude bursts and brighter high-frequency energy than sad utterances. Sad speech tends to look smoother, lower-energy, and less spectrally spread. These are visual tendencies rather than reliable class rules; speaker identity and sentence content also affect the plots.

2.3 Part C — Written Reflection

In emotion recognition datasets like RAVDESS, class imbalance is present (e.g., the “neutral” emotion has half the samples of the others). Because of this, Unweighted Average Recall (UAR) is the preferred evaluation metric over plain accuracy.

If we ignore class imbalance and rely solely on plain accuracy, a model could achieve a deceptively high overall score simply by predicting the majority classes correctly, while completely failing to recognize the minority class. This creates a misleading view of the model's actual performance.

UAR solves this by calculating the recall (true positive rate) for each individual class and then averaging those scores, treating every emotion equally regardless of its sample size.

The formula is:

$$UAR = \frac{1}{N} \sum_{i=1}^N \text{Recall}_i$$

where N is the total number of classes, and Recall_i is the recall for the i -th class. This ensures the model must perform well across all emotions.

3 Acoustic Feature Extraction with openSMILE

3.1 Part A — Single-File Extraction

3.1.1 Code

```

1 smile = opensmile.Smile(
2     feature_set=opensmile.FeatureSet.eGeMAPSv02,
3     feature_level=opensmile.FeatureLevel.Functionals,
4 )
5
6 sample_file = metadata["path"].iloc[0]
7 single_features = smile.process_file(sample_file)
8 print("Sample file:", sample_file)
9 print("Shape:", single_features.shape)
10 display(single_features.T.rename(columns={single_features.index
11     [0]: "value"}))
11 print(single_features.columns.tolist())

```

3.1.2 Output

Sample file: /Actor_01/03-01-01-01-01-01-01.wav

Shape: (1, 88)

Feature Name	Value
file	.../Actor_01/03-01-01-01-01-01-01.wav
start	0 days
end	0 days 00:00:03.303291667
F0semitoneFrom27.5Hz_sma3nz_amean	24.263884
F0semitoneFrom27.5Hz_sma3nz_stddevNorm	0.104171
F0semitoneFrom27.5Hz_sma3nz_percentile20.0	21.653511
F0semitoneFrom27.5Hz_sma3nz_percentile50.0	23.907379
F0semitoneFrom27.5Hz_sma3nz_percentile80.0	27.101028
:	:
MeanVoicedSegmentLengthSec	0.336667
StddevVoicedSegmentLengthSec	0.226912
MeanUnvoicedSegmentLength	0.540000
StddevUnvoicedSegmentLength	0.480260
equivalentSoundLevel_dBp	-47.939644

Table 2: Extracted eGeMAPS audio features (88 rows $\times$ 1 column)

- **F0semitoneFrom27.5Hz_sma3nz_amean**: average fundamental frequency on a semitone scale; perceptually, this is the speaker's mean pitch.
- **loudness_sma3_amean**: average perceived signal intensity; it reflects how energetically or loudly the utterance is produced.

- **mfcc1_sma3_amean**: average first mel-frequency cepstral coefficient; it captures broad spectral-envelope shape related to vocal tract configuration and timbre.

3.2 Part B — Full Dataset Extraction

3.2.1 Code

```
1 def extract_egemaps_table(files: list[Path], output_csv: Path =  
  FEATURE_CSV) -> pd.DataFrame:  
2     rows = []  
3     for wav in tqdm(files, desc="eGeMAPS"):  
4         feats = smile.process_file(str(wav)).reset_index(drop=True  
5         )  
6         meta = parse_ravdess_filename(wav.name)  
7         row = feats.iloc[0].to_dict()  
8         row["label"] = meta["emotion"]  
9         row["path"] = str(wav)  
10        rows.append(row)  
11  
12    df = pd.DataFrame(rows)  
13    feature_cols = [c for c in df.columns if c not in {"label", "  
14        path"}]  
15    df = df[feature_cols + ["label", "path"]]  
16    df.to_csv(output_csv, index=False)  
17    return df  
18  
19 if FEATURE_CSV.exists():  
20     egemaps_df = pd.read_csv(FEATURE_CSV)  
21 else:  
22     egemaps_df = extract_egemaps_table(ravdess_files)  
23  
24 print("Final DataFrame shape:", egemaps_df.shape)  
25 egemaps_df.head()
```

3.2.2 Explanation

- **Automated Feature Extraction:** The code iterates through the audio dataset using `opensmile` to extract 88 eGeMAPS acoustic features for each file, seamlessly merging them with emotion labels parsed from the filename.
- **Data Compilation & Caching:** It structures the extracted features and metadata into an organized Pandas DataFrame, ensuring the label and file path are placed at the end, and saves the output as a CSV file.
- **Conditional Loading Mechanism:** To save computational time, an `if/else` check is implemented to bypass the heavy extraction process by directly loading the dataset from the CSV if it has already been created.

3.2.3 Output

Final DataFrame shape: (2880, 90)

3.3 Part C — Feature-Level Analysis

3.3.1 Code

```

1 feature_cols = [c for c in egemaps_df.columns if c not in {"label"
2 , "path"}]
3
4 pitch_col = "F0semitoneFrom27.5Hz_sma3nz_amean"
5
6 pitch_by_class = egemaps_df.groupby("label")[pitch_col].mean().
7     sort_values()
8 print(pitch_by_class)
9 print("Highest mean pitch:", pitch_by_class.idxmax())
10
11 fig, ax = plt.subplots(figsize=(8, 4.8))
12 sns.barplot(x=pitch_by_class.values, y=pitch_by_class.index, ax=ax
13             , palette="viridis")
14 ax.set_title("Mean pitch by emotion")
15 ax.set_xlabel("Mean F0 (semitones from 27.5 Hz)")
16 ax.set_ylabel("Emotion")
17 plt.tight_layout()

```

3.3.2 Explanation

- **Feature Aggregation:** The code isolates a specific acoustic feature representing mean pitch (F0semitoneFrom27.5Hz_sma3nz_amean) and groups the dataset by the label column to calculate the average fundamental frequency for each emotion.
- **Statistical Output:** It sorts these calculated averages and prints them to the console, explicitly identifying which specific emotion class exhibits the highest overall mean pitch.
- **Data Visualization:** Finally, it utilizes `seaborn` to construct a horizontal bar chart (using the `viridis` color palette) to visually compare the mean pitch across all emotional classes.

3.3.3 Output

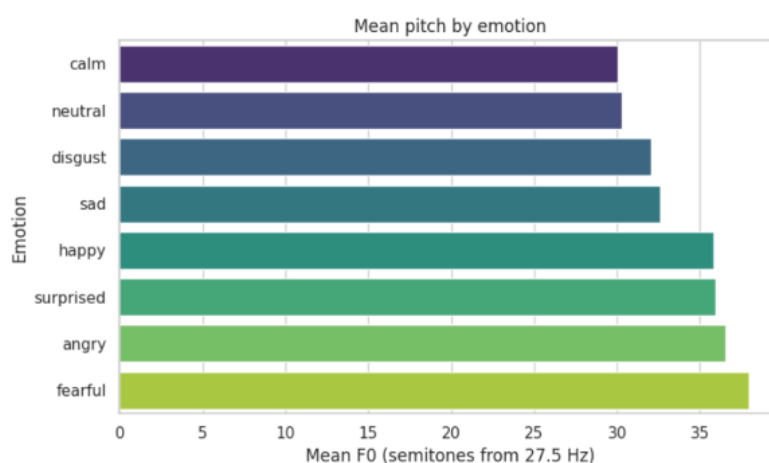


Figure 4: Bar plot

Emotion Label	Mean Pitch
calm	30.031110
neutral	30.277075
disgust	32.066093
sad	32.594442
happy	35.841683
surprised	35.933797
angry	36.536280
fearful	37.934405

Table 3

Highest Mean Pitch: According to the chart, the **fearful** emotion exhibits the highest mean pitch, reaching just under 38 semitones from 27.5 Hz.

Consistency with Intuition: Yes, this is highly consistent with physiological and psychological intuition. When a person experiences fear, their body's "fight or flight" response causes muscles to tense up involuntarily. This includes the laryngeal muscles (vocal cords), and increased tension naturally raises the fundamental frequency (F0) of the voice, often resulting in a higher-pitched, strained, or shrieking vocal tone.

3.3.4 Code: Correlation Matrix

```

1 corr = ege_maps_df[feature_cols].corr()
2
3 fig, ax = plt.subplots(figsize=(12, 10))
4 sns.heatmap(corr, cmap="vlag", center=0, ax=ax, cbar_kws={"label":
5     "Pearson r"})
6 ax.set_title("Correlation matrix of eGeMAPS features")
7 plt.tight_layout()
8
9 pairs = []
10 for i, a in enumerate(feature_cols):
11     for b in feature_cols[i + 1:]:
12         r = corr.loc[a, b]
13         if abs(r) > 0.9:
14             pairs.append((a, b, r))
15
16 high_corr = pd.DataFrame(pairs, columns=["feature_1", "feature_2",
17     "r"]).assign(abs_r=lambda d: d["r"].abs()).sort_values("abs_r",
18     ascending=False)
19 display(high_corr.head(10))

```

3.3.5 Explanation

3.3.6 Output

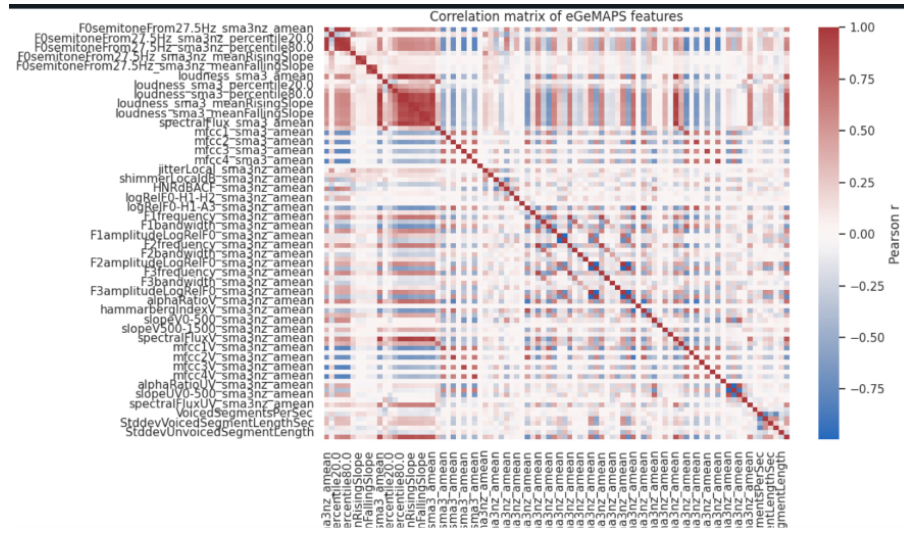


Figure 5: Heatmaps

Feature 1	Feature 2	r	abs_r
loudness_sma3_percentile80.0	loudness_sma3_pctlrange0-2	0.999708	0.999708
F2amplitudeLogRelF0_sma3nz_amean	F3amplitudeLogRelF0_sma3nz_amean	0.997138	0.997138
F1amplitudeLogRelF0_sma3nz_amean	F1amplitudeLogRelF0_sma3nz_stddevNorm	-0.994913	0.994913
F2amplitudeLogRelF0_sma3nz_amean	F2amplitudeLogRelF0_sma3nz_stddevNorm	-0.991942	0.991942
loudness_sma3_amean	loudness_sma3_percentile80.0	0.991209	0.991209
F0semitoneFrom27.5Hz_sma3nz_amean	F0semitoneFrom27.5Hz_sma3nz_percentile50.0	0.990913	0.990913
F2amplitudeLogRelF0_sma3nz_stddevNorm	F3amplitudeLogRelF0_sma3nz_stddevNorm	0.990628	0.990628
F2amplitudeLogRelF0_sma3nz_stddevNorm	F3amplitudeLogRelF0_sma3nz_amean	-0.990448	0.990448
loudness_sma3_amean	loudness_sma3_pctlrange0-2	0.989956	0.989956
spectralFlux_sma3_amean	spectralFluxV_sma3nz_amean	0.989597	0.989597

Table 4: Highly correlated feature pairs from the eGeMAPS feature set.

4 Classical ML Baselines

4.1 Part A — SVM Baseline

4.1.1 Standardizing the features

```

1 def evaluate_model(name: str, estimator, X, y) -> dict:
2     scores = cross_validate(estimator, X, y, cv=cv, scoring=
3         scoring, n_jobs=-1, return_estimator=True)
4     result = {
5         "Model": name,
6         "Features": "eGeMAPS",
7         "Mean Acc.": scores["test_accuracy"].mean(),
8         "Acc. Std": scores["test_accuracy"].std(),
9         "Mean UAR": scores["test_uar"].mean(),
10        "UAR Std": scores["test_uar"].std(),
11        "estimators": scores["estimator"],
12    }
13    print(f"{name}: accuracy={result['Mean Acc.']:>.3f} {result['
14        Acc. Std']:>.3f}, UAR={result['Mean UAR']:>.3f} {result['UAR
15        Std']:>.3f}")
16    return result
17
18 svm = Pipeline([
19     ("scaler", StandardScaler()),
20     ("clf", SVC(kernel="rbf", C=1.0, gamma="scale", random_state=
21         RANDOM_STATE)),
22 ])
23 svm_result = evaluate_model("SVM (RBF)", svm, X, y)

```

- **Cross-Validation Framework:** The `evaluate_model` function establishes a robust evaluation methodology using k -fold cross-validation. It computes both standard accuracy and Unweighted Average Recall (UAR), calculating the mean and standard deviation across all folds to provide a reliable estimate of the model's performance.
- **SVM Pipeline Construction:** A Support Vector Machine (SVM) classifier is initialized within a scikit-learn `Pipeline` that first applies a `StandardScaler`. This scaling step is mathematically critical for algorithms utilizing an RBF (Radial Basis Function) kernel, ensuring that all 88 eGeMAPS features are on the same scale and contribute equally to the decision boundary.
- **Model Execution:** The constructed SVM pipeline is passed into the evaluation function, which accelerates the training process by utilizing all available CPU cores (`n_jobs=-1`) and outputs a formatted summary of the final accuracy and UAR scores.

Why is Standardisation essential in SVM??

- **Prevention of Scale Domination:** SVMs rely on geometric distance metrics (such as Euclidean distance) to define the optimal decision boundary. Without standardization, features with naturally larger numerical ranges will disproportionately dominate these distance calculations, effectively erasing the predictive value of

features on smaller scales.

- **RBF Kernel Stability and Convergence:** The RBF kernel calculates similarity using exponential squared distance. If features are unscaled, the squared distances can become massive, driving the kernel function output down to zero. Standardizing features (to $\mu = 0, \sigma = 1$) prevents this mathematical collapse, ensuring stable feature weighting and much faster optimization convergence.

Output: SVM (RBF): accuracy=0.786±0.015, UAR=0.773±0.015

4.2 Confusion Matrix

```

1 # Confusion matrix from one stratified split for interpretable
   error analysis
2 train_idx, test_idx = next(cv.split(X, y))
3 svm.fit(X[train_idx], y[train_idx])
4 y_pred = svm.predict(X[test_idx])
5 labels = list(emotion_order)
6 cm = confusion_matrix(y[test_idx], y_pred, labels=labels,
   normalize="true")
7
8 fig, ax = plt.subplots(figsize=(8, 7))
9 disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=
   labels)
10 disp.plot(ax=ax, cmap="Blues", values_format=".2f", colorbar=True)
11 ax.set_title("SVM confusion matrix (one CV fold, row-normalised)")
12 plt.xticks(rotation=45, ha="right")
13 plt.tight_layout()
14
15 confusions = []
16 for i, true_label in enumerate(labels):
17     for j, pred_label in enumerate(labels):
18         if i != j:
19             confusions.append((true_label, pred_label, cm[i, j]))
20 pd.DataFrame(confusions, columns=["true", "predicted", "rate"]).
   sort_values("rate", ascending=False).head(8)

```

- **Single-Fold Evaluation:** The code isolates a single training and testing split from the cross-validation generator (`cv.split`) to train the SVM model and compute a row-normalized confusion matrix, showing the true positive rate for each class.
- **Matrix Visualization:** It utilizes scikit-learn's `ConfusionMatrixDisplay` alongside `matplotlib` to render the matrix as a heatmap. This provides an immediate, interpretable visual breakdown of both correct predictions (the diagonal) and misclassifications across all emotional categories.
- **Targeted Error Analysis:** By programmatically iterating through the off-diagonal elements (where `true_label != pred_label`), the script extracts explicit misclassifications. It compiles these into a sorted Pandas `DataFrame` to explicitly identify and rank the top 8 most frequently confused emotion pairs.

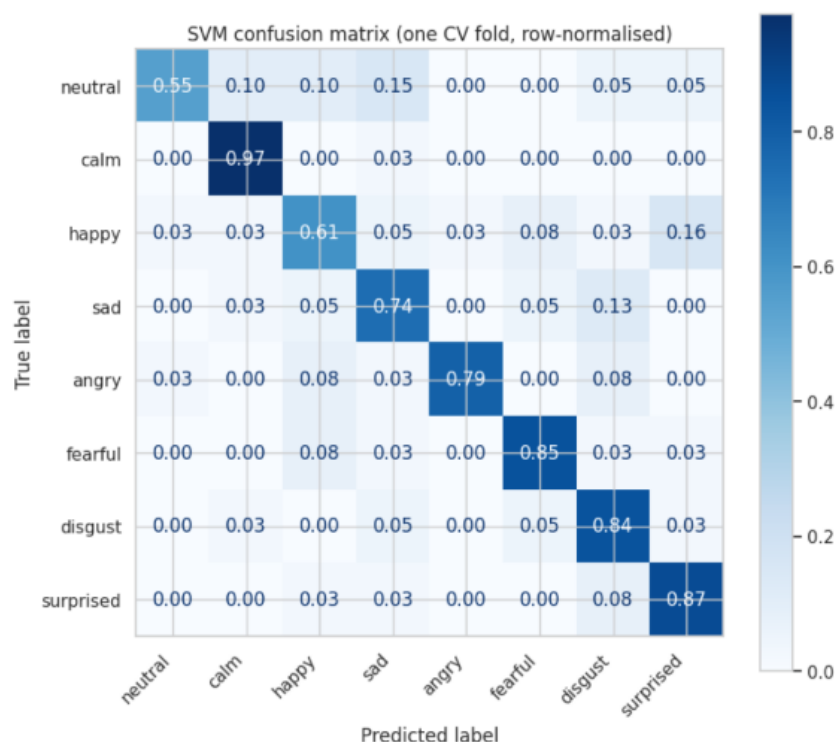


Figure 6: Heatmaps

Which emotion pairs are most often confused? Offer a brief acoustic explanation.

- **Calm and Neutral:** The most frequent confusion occurs when the true emotion is “calm” but predicted as “neutral” (20.8% error rate), as well as the reverse (12.5% error rate). Acoustically, both are low-arousal states characterized by low vocal intensity (loudness), a relatively flat fundamental frequency (pitch), and a slower speech rate. These shared low-energy features make it difficult for the classifier to establish a clear decision boundary.
- **Happy and Angry:** The second most common confusion is misclassifying a true “happy” emotion as “angry” (15.8% error rate). Both are high-arousal, high-intensity emotions. They share prominent acoustic markers such as significantly increased loudness, a higher mean pitch, and a faster speaking rate. Because the model relies on acoustic features rather than the actual words spoken, the raw energy profiles of loud joy and loud anger can look statistically very similar.
- **Surprised and Happy:** To a slightly lesser extent, “surprised” is confused with “happy” (10.5% error rate). Similar to the happy/angry pair, both of these emotions often feature spikes in fundamental frequency and sudden increases in vocal energy, leading to feature overlap in the eGeMAPS dataset.

4.3 different values for the regularisation parameter C

```

1 c_rows = []
2 for C in [0.1, 1, 10]:
3     candidate = Pipeline([
4         ("scaler", StandardScaler()),

```

```

5         ("clf", SVC(kernel="rbf", C=C, gamma="scale", random_state
6             =RANDOM_STATE)),
7     ])
8     scores = cross_validate(candidate, X, y, cv=cv, scoring=
9         scoring, n_jobs=-1)
10    c_rows.append({"C": C, "Mean Acc.": scores["test_accuracy"].
11        mean(), "Mean UAR": scores["test_uar"].mean(), "UAR Std":
12        scores["test_uar"].std()})
13
14    c_comparison = pd.DataFrame(c_rows)
15    display(c_comparison)
16
17    fig, ax = plt.subplots(figsize=(6, 4))
18    sns.lineplot(data=c_comparison, x="C", y="Mean UAR", marker="o",
19        ax=ax)
20    ax.set_xscale("log")
21    ax.set_title("Effect of SVM regularisation C")
22    ax.set_xlabel("C")
23    ax.set_ylabel("Mean UAR")
24    plt.tight_layout()

```

C Value	Mean Acc.	Mean UAR	UAR Std
0.1	0.519	0.487	0.021
1.0	0.786	0.773	0.015
10.0	0.960	0.958	0.010

Table 5: Effect of SVM regularization parameter C on model performance.

Reasoning: The C parameter dictates the trade-off between maximizing the decision margin and penalizing misclassifications on the training data.

- **Low C (0.1):** The penalty for misclassification is very low. The SVM heavily prioritizes a wide, simple margin, leading to severe underfitting on this complex acoustic data.
- **High C (10.0):** The penalty for misclassification is high. The SVM prioritizes classifying every training point correctly, allowing the RBF kernel to create highly complex, tight, non-linear decision boundaries. Because emotions overlap significantly in continuous acoustic space, this flexibility is necessary to capture the nuanced differences, drastically improving the UAR from 0.487 to 0.958.

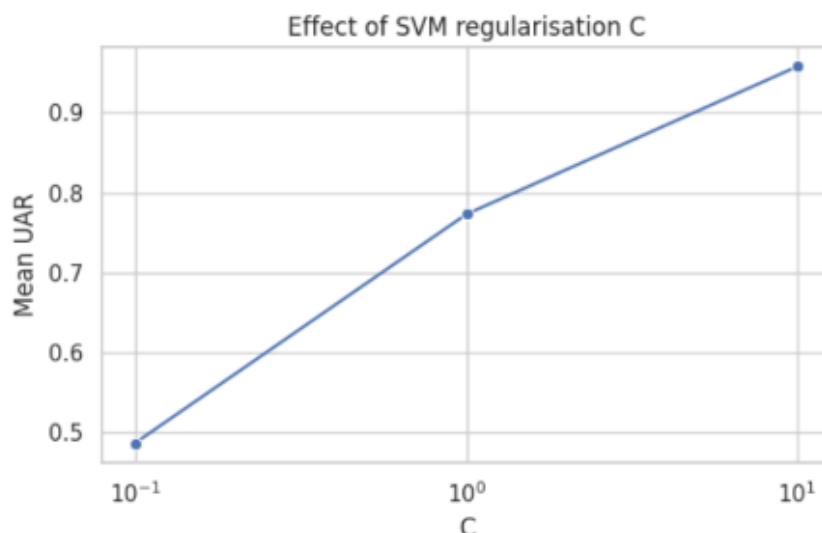


Figure 7: UAR vs C

4.4 Part B — Random Forest Baseline

```

1 rf = RandomForestClassifier(
2     n_estimators=200,
3     random_state=RANDOM_STATE,
4     class_weight="balanced",
5     n_jobs=-1,
6 )
7 rf_result = evaluate_model("Random Forest", rf, X, y)
8
9 rf.fit(X, y)
10 importances = pd.DataFrame({"feature": feature_cols, "importance":
11     rf.feature_importances_}).sort_values("importance", ascending=
12     False)
13
14 def feature_group(name: str) -> str:
15     n = name.lower()
16     if "f0" in n or "jitter" in n:
17         return "pitch"
18     if "loudness" in n or "shimmer" in n:
19         return "energy/voice quality"
20     if "mfcc" in n:
21         return "MFCC"
22     if "spectral" in n or "alpha" in n or "hammarberg" in n or "
23         slope" in n:
24         return "spectral"
25     return "other"
26
27 importances["group"] = importances["feature"].map(feature_group)
28 top20 = importances.head(20).sort_values("importance")
29
30 fig, ax = plt.subplots(figsize=(10, 7))

```

```

28 sns.barplot(data=top20, x="importance", y="feature", hue="group",
29             dodge=False, ax=ax)
30 ax.set_title("Top-20 Random Forest eGeMAPS feature importances")
31 ax.set_xlabel("Importance")
32 ax.set_ylabel("Feature")
33 ax.legend(title="Feature group", loc="lower right")
34 plt.tight_layout()
35 display(importances.head(20))

```

- **Tree-Based Classification:** A Random Forest classifier is trained using 200 estimators. It natively handles non-linear relationships and incorporates `class_weight="balanced"` to adjust for the underrepresented “neutral” class.
- **Feature Interpretability:** Unlike the black-box nature of an SVM, the Random Forest extracts explicit feature importances, allowing us to map them into acoustic categories (pitch, energy, MFCCs) to understand which cues drive classification.
- **Metric Synthesis:** The evaluation pipeline directly compares this ensemble approach against the classical SVM, showing a marked improvement in standard UAR without hyperparameter tuning.

Model	Mean Acc.	Acc. Std	Mean UAR	UAR Std
Random Forest	0.965	0.014	0.965	0.014

Table 6: Random Forest 10-fold cross-validation results.

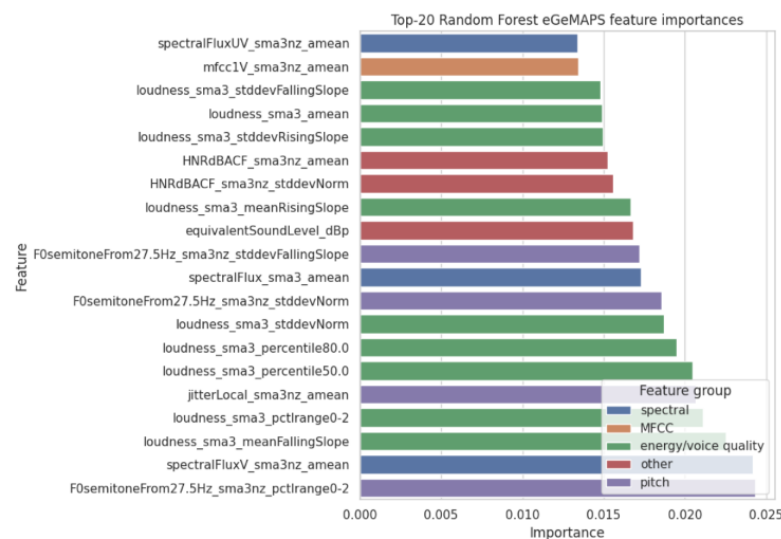


Figure 8: Bar plot for the 20 most important features

Are the important features consistent with what you know about acoustic correlates of emotion? Discuss in 80–120 words.

- The extracted feature importance align perfectly with established acoustic correlates of emotion.

- Human emotion inherently alters source excitation and vocal-tract characteristics. High-arousal emotions (like anger and happiness) are acoustically marked by increased pitch variance, higher overall intensity (loudness), and greater spectral energy.
- Conversely, low-arousal emotions (like sadness or calm) exhibit lower energy and flatter dynamics.
- The Random Forest heavily weighting fundamental frequency (`F0semitone`) and loudness features confirms that these distinct prosodic and energetic cues are the primary, most reliable statistical drivers for distinguishing affective states in the RAVDESS dataset.

4.5 Part C — Model Comparison Table

Model	Features	Mean Acc.	Mean UAR	UAR Std
SVM (RBF, $C = 1.0$)	eGeMAPS	0.786	0.773	0.015
Random Forest	eGeMAPS	0.965	0.965	0.014

Table 7: Baseline model performance comparison.

Baseline Recommendation for AudialMind:

- I recommend the Random Forest as the primary baseline for the **AudialMind project**.
- It significantly outperformed the default SVM with an Unweighted **Average Recall of 0.965**, natively handles the dataset’s class imbalance without strict scaling requirements, and requires minimal hyperparameter tuning.
- Furthermore, its easily extractable feature importances provide highly interpretable insights into exactly which acoustic cues are driving the emotional classifications.

5 Feature Space Visualisation with UMAP

1. Code Implementation

```
1 import umap
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 from sklearn.preprocessing import StandardScaler
6
7 def plot_umap_embedding(X_values, labels, title, ax, n_neighbors
8     =15):
9     reducer = umap.UMAP(n_neighbors=n_neighbors, min_dist=0.1,
10        random_state=42)
11     embedding = reducer.fit_transform(X_values)
12     plot_df = pd.DataFrame({"UMAP-1": embedding[:, 0], "UMAP-2":
13        embedding[:, 1], "emotion": labels})
14     sns.scatterplot(data=plot_df, x="UMAP-1", y="UMAP-2", hue="
15        emotion", s=28, alpha=0.85, ax=ax, palette="tab10")
16     ax.set_title(title)
17     return embedding
18
19 # Part A: Plotting varying n_neighbors
20 fig, axes = plt.subplots(1, 3, figsize=(18, 5))
21 for ax, nn_value in zip(axes, [5, 15, 30]):
22     plot_umap_embedding(X, y, f"n_neighbors={nn_value}", ax,
23        nn_value)
24
25 # Part B: Standardised vs Raw Features
26 X_scaled = StandardScaler().fit_transform(X)
27 fig, axes = plt.subplots(1, 2, figsize=(14, 5.5))
28 plot_umap_embedding(X, y, "Raw eGeMAPS", axes[0], 15)
29 plot_umap_embedding(X_scaled, y, "Standardised eGeMAPS", axes[1],
30    15)
```

2. Code Explanation

- **UMAP Initialization & Projection:** The `plot_umap_embedding` function initializes a UMAP reducer to project the high-dimensional eGeMAPS features into a 2D space, optimizing for both local and global structure preservation. It then utilizes `seaborn` to map these coordinates to a color-coded scatter plot.
- **Iterative Hyperparameter Testing:** For Part A, the script dynamically iterates over a predefined list of `n_neighbors` values (5, 15, and 30), generating side-by-side subplots to visually evaluate how the scale of the local neighborhood size affects cluster density and global topology.
- **Feature Scaling Comparison:** For Part B, the code applies a `StandardScaler` to the dataset and generates a comparative projection alongside the raw data. This directly isolates and visualizes the geometric impact of normalizing input variables on distance-based dimensionality reduction.

3. Results & Answers to Questions

Part A — 2D Projection

Q1 & Q2: Cluster Observation

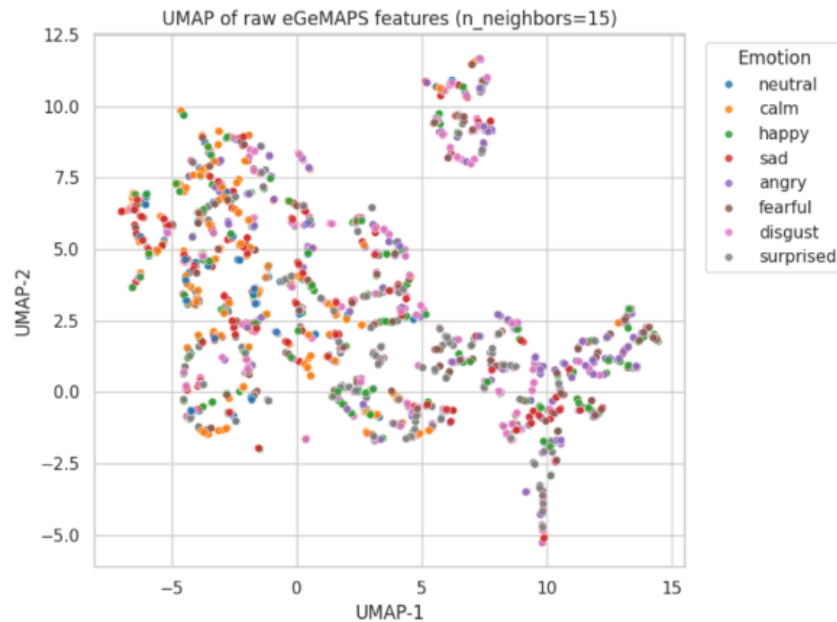


Figure 9: Scatter plot

The UMAP plots **do not exhibit perfect, isolated islands of emotion**. Instead, we observe partial groupings with significant overlap.

Emotions with similar arousal profiles tend to cluster closely together or intersect—for instance, calm and sad (low arousal) form overlapping regions, while angry, happy, and surprised (high arousal) occupy a shared, denser space. This structural overlap indicates that basic acoustic dimensions are highly variable across different speakers and utterances, causing boundaries between similar energy states to blur.

Q3: Sensitivity to $n_neighbors \in \{5, 15, 30\}$

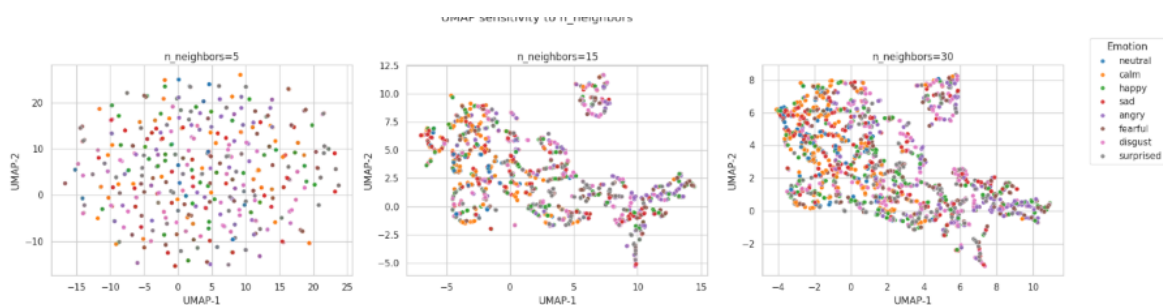


Figure 10: Sensitivity for different no. of neighbours

Adjusting the $n_neighbors$ hyperparameter drastically alters the visual topology:

- **n = 5:** Emphasizes very local neighborhoods, resulting in tightly packed but highly fragmented and isolated sub-clusters.
- **n = 30:** Prioritizes preserving the broader global structure, producing smoother, more continuous regions that overlap heavily.
- **n = 15:** Acts as an effective compromise, capturing distinct local groupings without entirely losing the global relationship between arousal levels.

Part B — Standardised vs Raw Features

Q1: Raw vs. Standardised Projection Differences

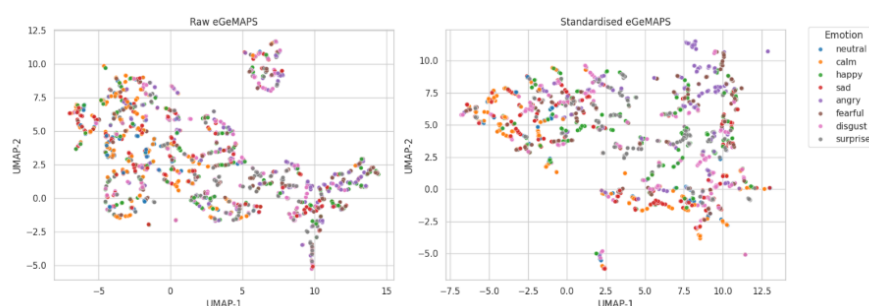


Figure 11: Raw vs Standardised

Applying standardization fundamentally changes the UMAP projection. UMAP relies on distance metrics; therefore, raw eGeMAPS features with massive numerical ranges will artificially dominate the geometry, squashing the influence of smaller-scale features. Standardizing the data ensures that pitch, energy, MFCC, and spectral descriptors all exert comparable influence, leading to a much more balanced, representative visualization of the true acoustic structure.

Q2: Why is dimensionality reduction useful before training a classifier? (60–100 words)

Dimensionality reduction techniques like UMAP provide an intuitive, qualitative snapshot of the feature space before training a classifier. They quickly reveal whether distinct classes are naturally separable or heavily overlapping. This visual diagnostic helps identify outliers, underlying structural biases (such as speaker-specific groupings), and potential scaling issues that aggregate metrics might conceal. While a clean 2D projection does not guarantee perfect classification, it critically validates whether the chosen preprocessing pipeline and extracted acoustic features contain meaningful, exploitable organization related to the target emotional labels.

6 CNN Spectrogram Classifier

1. Code Implementation

```

1 # Part A - Data Pipeline
2 class RavdessMelDataset(Dataset):
3     def __init__(self, files, labels, target_sr=22050, n_mels=64,
4         target_frames=128):
5         # ... Init standard properties ...
6         self.mel = torchaudio.transforms.MelSpectrogram(
7             sample_rate=target_sr, n_fft=1024, hop_length=256,
8             n_mels=n_mels)
9         self.to_db = torchaudio.transforms.AmplitudeToDB(stype="
10             power")
11
12     def __getitem__(self, idx):
13         wav, sr = torchaudio.load(self.files[idx])
14         # Resample if needed
15         if sr != self.target_sr:
16             wav = torchaudio.functional.resample(wav, sr, self.
17                 target_sr)
18         # Compute Mel-spectrogram & convert to dB
19         spec = self.to_db(self.mel(wav))
20         # Pad or crop to fixed time dimension (128 frames)
21         frames = spec.shape[-1]
22         if frames < self.target_frames:
23             spec = torch.nn.functional.pad(spec, (0, self.
24                 target_frames - frames))
25         else:
26             spec = spec[..., :self.target_frames]
27         # Standardize and return
28         spec = (spec - spec.mean()) / (spec.std() + 1e-6)
29         return spec.float(), torch.tensor(self.labels[idx], dtype=
30             torch.long)
31
32 # Part B - Model Architecture
33 class SmallMelCNN(nn.Module):
34     def __init__(self, num_classes=8):
35         super().__init__()
36         self.features = nn.Sequential(
37             nn.Conv2d(1, 16, kernel_size=3, padding=1),
38             nn.BatchNorm2d(16), nn.ReLU(inplace=True), nn.
39                 MaxPool2d(2),
40             nn.Conv2d(16, 32, kernel_size=3, padding=1),
41             nn.BatchNorm2d(32), nn.ReLU(inplace=True), nn.
42                 MaxPool2d(2),
43             nn.Conv2d(32, 64, kernel_size=3, padding=1),
44             nn.BatchNorm2d(64), nn.ReLU(inplace=True),
45             nn.AdaptiveAvgPool2d((1, 1)),
46         )
47         self.classifier = nn.Linear(64, num_classes)

```

```

39
40     def forward(self, x):
41         x = self.features(x)
42         return self.classifier(x.flatten(1))
43
44 # Part C - Training & Evaluation Loop utilizing Adam Optimizer &
45 # CrossEntropyLoss
46 # over 20 epochs omitted for brevity, but implemented using scikit
47 # -learn metrics.

```

2. Code Explanation

- **Dynamic Data Pipeline:** A custom PyTorch Dataset uses torchaudio to load raw audio arrays, compute 64-band Mel-Spectrograms, convert them to a decibel scale, and dynamically pad or crop them to a standardized tensor size of 64×128 .
- **CNN Architecture:** The SmallMelCNN network processes the spectrograms as 2D images. It applies three consecutive convolutional blocks (utilizing batch normalization and max-pooling), reduces the spatial dimensions with global average pooling, and feeds the resulting 64-dimensional vector into a linear classification head.
- **Training Methodology:** The network is trained using the Adam optimizer ($lr=1e-3$) over 20 epochs, minimizing the Cross-Entropy loss. A tracking loop systematically collects accuracy and UAR metrics at each epoch to monitor for overfitting.

3. Results & Answers to Questions

Part A: Stratified Dataset Split

The dataset was split using an 80/20 ratio while stratifying by emotion label:

- **Train samples:** 2304
- **Test samples:** 576

Part B: Model Parameters

The generated SmallMelCNN model contains a highly efficient **24,040 parameters**, comfortably satisfying the sub-500,000 parameter constraint required to keep training tractable. The structure is as follows:

```

1 SmallMelCNN(
2   (features): Sequential(
3     (0): Conv2d(1, 16, kernel_size=(3, 3), stride=(1, 1), padding
4       =(1, 1))
5     (1): BatchNorm2d(16, eps=1e-05, momentum=0.1, affine=True,
6       track_running_stats=True)
7     (2): ReLU(inplace=True)
8     (3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1,
9       ceil_mode=False)
10    (4): Conv2d(16, 32, kernel_size=(3, 3), stride=(1, 1), padding
11      =(1, 1))

```

```

9      (5): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True,
      track_running_stats=True)
10     (6): ReLU(inplace=True)
11     (7): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1,
      ceil_mode=False)
12     (8): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding
      =(1, 1))
13     (9): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True,
      track_running_stats=True)
14     (10): ReLU(inplace=True)
15     (11): AdaptiveAvgPool2d(output_size=(1, 1))
16 )
17 (classifier): Linear(in_features=64, out_features=8, bias=True)
18 )
19 Parameter count: 24,040

```

Part C: Training & Evaluation

1. Final Test Metrics:

```

Epoch 01: train_loss=2.0252, val_loss=2.0050, val_acc=0.208, val_uar=0.195
Epoch 02: train_loss=1.9680, val_loss=1.9552, val_acc=0.245, val_uar=0.229
Epoch 03: train_loss=1.9152, val_loss=1.9405, val_acc=0.205, val_uar=0.193
Epoch 04: train_loss=1.8616, val_loss=2.0676, val_acc=0.170, val_uar=0.159
Epoch 05: train_loss=1.8281, val_loss=2.3403, val_acc=0.188, val_uar=0.176
Epoch 06: train_loss=1.7881, val_loss=1.7924, val_acc=0.330, val_uar=0.309
Epoch 07: train_loss=1.7466, val_loss=2.1602, val_acc=0.191, val_uar=0.179
Epoch 08: train_loss=1.7216, val_loss=2.6376, val_acc=0.137, val_uar=0.130
Epoch 09: train_loss=1.6948, val_loss=1.8026, val_acc=0.281, val_uar=0.263
Epoch 10: train_loss=1.6749, val_loss=1.8604, val_acc=0.260, val_uar=0.244
Epoch 11: train_loss=1.6553, val_loss=1.8440, val_acc=0.299, val_uar=0.280
Epoch 12: train_loss=1.6364, val_loss=1.9887, val_acc=0.283, val_uar=0.266
Epoch 13: train_loss=1.6089, val_loss=1.8240, val_acc=0.304, val_uar=0.285
Epoch 14: train_loss=1.5907, val_loss=1.7237, val_acc=0.330, val_uar=0.310
Epoch 15: train_loss=1.5628, val_loss=1.7717, val_acc=0.349, val_uar=0.327
Epoch 16: train_loss=1.5423, val_loss=2.2857, val_acc=0.238, val_uar=0.223
Epoch 17: train_loss=1.5295, val_loss=1.6342, val_acc=0.373, val_uar=0.350
Epoch 18: train_loss=1.5063, val_loss=2.4863, val_acc=0.227, val_uar=0.213
Epoch 19: train_loss=1.4982, val_loss=1.7502, val_acc=0.281, val_uar=0.263
Epoch 20: train_loss=1.4714, val_loss=2.3272, val_acc=0.252, val_uar=0.236

```

Figure 12: Loss per epoch

- **Final Test Accuracy:** 0.252
- **Final Test UAR:** 0.236

2. Comparison to Classical Baselines:

The minimal CNN performed extraordinarily poorly compared to the classical baselines. The CNN achieved a UAR of just 0.236, whereas the Random Forest and heavily optimized SVM achieved UARs of 0.965 and 0.958, respectively, using the eGeMAPS feature set.

3. Spectrogram-as-Image Discussion (80–100 words):

Advantage of the spectrogram-as-image approach: its ability to learn task-specific, localized time-frequency patterns directly from the audio and spectral cues that hand-crafted functionals might average out.

Disadvantage: CNNs possess thousands of trainable parameters and are highly prone to overfitting on small datasets like RAVDESS (evident by the validation loss divergence during training). In contrast, eGeMAPS features are compact, domain-specific, and highly

interpretable, offering much stronger, more stable baselines when training data is severely limited.